Disciplina: Sistemas Distribuídos
Professora: Ana Cristina Barreiras Kochem Vendramin

Avaliação (valor 2,0)
Microsserviços, Sistema de Mensageria, Criptografia de Chave Assimétrica

Desenvolver um sistema distribuído baseado em **microsserviços** para gerenciamento e divulgação de **promoções de produtos**. O sistema deve seguir uma **arquitetura orientada a eventos (*Event-Driven Architecture*)**, na qual os microsserviços se comunicam **exclusivamente através de eventos publicados e consumidos em/de um *broker* RabbitMQ**. Cada microsserviço deverá atuar de forma **independente e desacoplada.** Então, não é permitido realizar chamadas diretas entre os microsserviços.

Usuários podem cadastrar promoções, votar em promoções cadastradas e receber notificações sobre promoções de seu interesse. Promoções com grande quantidade de votos positivos (estabelecer um limite) devem ser destacadas como **promoções em destaque (*hot deal*).**

(0,2) Processos Cliente Consumidores de Promoções

O **processo cliente consumidor** é responsável por receber notificações sobre promoções de interesse. Quando um usuário decide seguir uma determinada categoria de produto, ele passa a consumir notificações de promoções desta categoria no RabbitMQ.

(0,2) Cada cliente deve manifestar seu interesse em receber notificações de eventos sobre promoções de diferentes categorias e promoções em destaque. Por exemplo, um cliente interessado em promoções de livros, jogos e de promoções em destaque consumirá os eventos promocao.livro, promocao.jogo e promocao.destaque, respectivamente. Ao receber uma mensagem de notificação, esta será exibida no terminal.

Para simplificar, as categorias de interesse dos usuários podem estar definidas no código do cliente (*hard coded*).

O sistema deve utilizar uma exchange do tipo **direct** ou **topic** no RabbitMQ. Os eventos devem utilizar **routing keys hierárquicas**, permitindo que consumidores se inscrevam em diferentes categorias de eventos utilizando padrões de *binding*. Cada cliente pode criar sua própria fila e associá-la às *routing keys* correspondentes às categorias de interesse.

(1,8) Arquitetura baseada em eventos

O sistema deve ser implementado utilizando **4 microsserviços independentes**, cada um responsável por uma parte da lógica da aplicação. As responsabilidades desses microsserviços e os eventos publicados e/ou consumidos por cada um estão descritos a seguir.

1. **(0,5) Microsserviço *Gateway***

(0,2) Responsável por realizar a interação com os usuários (clientes e lojas) por meio do terminal. Ele apresenta as opções do sistema e permite que os usuários executem ações como cadastrar novas promoções, listar promoções publicadas (validadas pelo MS Promocao) e votar em promoções existentes. Esse serviço funciona como o ponto de entrada do sistema e transforma as ações realizadas pelos usuários em eventos que serão enviados para o *broker* RabbitMQ.

(0,1) Sempre que um evento for publicado, o *gateway* deve **gerar a assinatura digital da mensagem utilizando sua chave privada** e incluí-la no envelope do evento.

(0,2) O *gateway* **publica eventos** promocao.recebida, promocao.voto. Esse serviço **consome os eventos** promocao.publicada e mantém uma lista local de promoções validadas, permitindo que os usuários listem apenas promoções já aprovadas pelo microsserviço Promocao.

2. **(0,3) Microsserviço Promocao**

(0,2) Responsável pelo gerenciamento das promoções no sistema. Esse serviço recebe eventos indicando que novas promoções foram recebidas. Ao receber um evento, o serviço deve inicialmente **validar a assinatura digital da mensagem** para garantir sua autenticidade e integridade. Quando um evento de promoção é validado, o serviço registra a promoção, **assina e publica** um novo evento informando que a promoção foi disponibilizada no sistema.

(0,1) O microsserviço Promocao **consome os eventos** promocao.recebida, assina digitalmente e **publica eventos** promocao.publicada.

3. **(0,5) Microsserviço *Ranking***

(0,3) Responsável pelo processamento dos votos associados às promoções. Ao receber um evento, o serviço deve inicialmente **validar a assinatura digital da mensagem** para garantir sua autenticidade e integridade. Para cada evento validado, ele processa o voto (positivo ou negativo) da promoção correspondente, **atualiza o contador de votos e recalcula o *score* de popularidade da promoção específica**. Caso o *score* ultrapasse um limite definido pelo sistema, a promoção deve ser considerada uma promoção em destaque (*hot deal*). Quando isso ocorrer, o serviço **assina e publica** um novo evento indicando que a promoção foi destacada. Todos os eventos publicados por esse serviço devem ser assinados digitalmente antes de serem enviados ao RabbitMQ.

(0,2) O *ranking* **consome o evento** promocao.voto, assina digitalmente e **publica o evento** promocao.destaque.

4. **(0,5) Microsserviço Notificação**

(0,1) Responsável por distribuir notificações sobre promoções publicadas no sistema. Esse serviço consome eventos relacionados à publicação de novas promoções e à identificação de promoções em destaque. Ao receber um evento, o serviço deve **validar a assinatura digital da mensagem** para garantir sua autenticidade e integridade. Após a validação, o serviço identifica a categoria associada à promoção e publica uma notificação correspondente no RabbitMQ.

(0,4) Esse serviço **consome os eventos** promocao.publicada e promocao.destaque, e **publica eventos** promocao.categoria1, promocao.categoria2, ..., promocao.categoriaN. Para cada nova promoção em destaque, ele deve publicar um novo evento na categoria correspondente com a palavra "*hot deal*".

Assinatura digital dos eventos

Cada microsserviço (exceto Notificações) que **publicar um evento** deve:

1. gerar um *hash* do conteúdo do evento;
2. assinar o evento utilizando **criptografia assimétrica;**
3. incluir a assinatura digital no campo Signature.

A assinatura deve ser gerada utilizando a **chave privada do microsserviço produtor**.

Validação da assinatura

Todo microsserviço que **consumir um evento** deve:

1. verificar a assinatura digital utilizando a **chave pública do produtor;**
2. confirmar que o evento não foi alterado;
3. processar o evento **apenas se a assinatura for válida.** Caso a assinatura seja inválida, o evento deve ser **descartado**.

<u>Observações</u>:

- Desenvolva uma interface com recursos de interação apropriados.
- É obrigatória a defesa da aplicação para obter a nota.
- O desenvolvimento do sistema pode ser individual ou em dupla.

A Figura 1 ilustra os processos e a comunicação indireta entre eles.

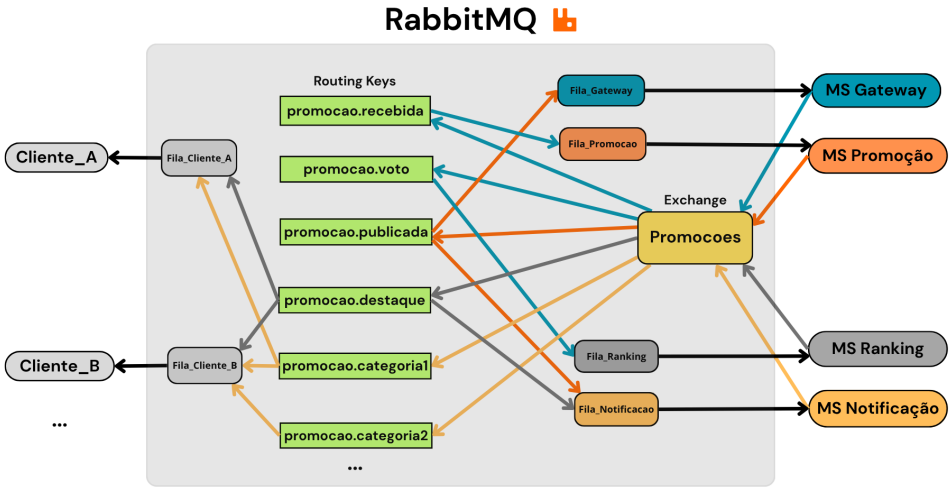

Figura 1 – Comunicação indireta entre os processos